

IBE : Orquestador de Agentes e Inteligencia Artificial

3 de agosto, 2025

Equipo y Roles:

- **Axel Joel Monge Ramírez** - Arquitecto en Jefe, Coordinador y Líder de Proyecto
- **Luis Elías Mora Montero** - Desarrollador Backend y de Agentes
- **Ignacio Serrano González** - Ingeniero de Hardware y Firmware

Descripción y Justificación

1. **Problema que se aborda:**

En la era actual de la inteligencia artificial, la promesa de una mayor productividad se ve opacada por una creciente fragmentación y complejidad. Los desarrolladores y usuarios avanzados se enfrentan a una avalancha de herramientas, APIs y agentes de IA desconectados, lo que convierte la integración en un desafío técnico y una barrera para la creatividad. Para el público general, incluyendo niños, adultos mayores o personas con discapacidades físicas, el acceso a este poder sigue siendo una utopía, limitado por la necesidad de interactuar con interfaces complejas y escribir código.

Vibe ataca este problema de raíz. Proponemos la siguiente revolución en la interacción humano-computadora: un nodo ciberfísico unificado (vestible como una muñequera o de escritorio como un asistente) que sirve como un orquestador central para el universo de la IA.

A través de comandos de voz simples e intuitivos, Vibe permite a cualquier persona:

- **Orquestar agentes de IA** complejos para controlar su computadora, gestionar su vida digital (email, calendario), realizar “vibe coding” mediante voz y automatizar flujos de trabajo.
- **Maximizar la eficiencia** proporcionando un asistente al estilo “JARVIS”, capaz de crear y monitorear los agentes de IA, crear comandos de voz reprogramables que ejecuten tareas indicadas por el usuario, y facilitar la interacción mediante una pantalla, proporcionando elementos gráficos cambiantes según el contexto.
- **Crear y personalizar sus propios agentes** sin escribir una sola línea de código, democratizando el acceso a la creación de herramientas de IA.
- **Interactuar con la tecnología de una manera fluida y natural**, eliminando la fricción entre la intención y la ejecución.

Vibe no es otro chatbot; es el puente tangible hacia la próxima generación que combina inteligencia artificial con sistemas ciberfísicos, combinando lo mejor de ambos mundos, y donde el poder de la IA se vuelve verdaderamente accesible, sencillo y moldeable para todos.

2. **Importancia y contexto:**

Vivimos en una era de explosión cámbrica de la inteligencia artificial. Cada día surgen nuevos modelos, APIs y agentes capaces de realizar tareas asombrosas. Este es el contexto: un ecosistema tecnológico inmensamente rico pero, a su vez, caótico y fragmentado. Para la mayoría de las personas, el acceso a este poder transformador está bloqueado por una barrera que sigue siendo abrumadoramente técnica.

Aquí radica la importancia de Vibe. No es simplemente otra aplicación de IA; es la capa de interacción unificada que faltaba. Vibe emerge como una solución de vital importancia al:

- **Democratizar el acceso:** Traduce la compleja orquestación de software en una simple conversación, permitiendo que emprendedores, creativos, y personas con discapacidades físicas puedan construir y comandar herramientas digitales sin ser expertos en programación.

- **Introducir un nuevo paradigma de interacción:** Al materializar la IA en un nodo ciberfísico, Vibe crea una relación fluida y tangible con la tecnología. Reduce la carga cognitiva, maximizando la eficiencia de la mayoría de tareas y haciendo que el control de sistemas complejos sea tan natural como hablar.
- **Potenciar la productividad:** Para los desarrolladores, Vibe elimina la fricción constante de cambiar entre terminales, editores y APIs, consolidando el flujo de trabajo de codificación en una única interfaz de voz inteligente y reprogramable.

En resumen, Vibe no solo aborda un problema de eficiencia, sino de accesibilidad universal. Su relevancia yace en su capacidad para ser el puente entre el potencial ilimitado de la IA y el ingenio humano, sin importar el nivel técnico del usuario.

3. **Usuarios/beneficiarios:**

El ecosistema de Vibe está diseñado para ser inmensamente inclusivo, aportando valor a un enorme espectro diverso de usuarios al democratizar el acceso a la automatización y la inteligencia artificial:

- **Desarrolladores y Profesionales de TI:** Son los beneficiarios principales. Vibe fue pensado principalmente como el mejor asistente no reemplazante de desarrolladores de software, como son los miembros del equipo de “Thinking Space”, la idea original descrita de forma ilustrativa, sería que cada desarrollador tuviera un “Intern/asistente” que automatice las tareas que el desarrollador decida sin comprometer el valor de “deep knowledge”, QA, supervisión, ajustes y creatividad de un humano real. Las acciones en las que pensamos incluyen gestionar las alarmas y correos del usuario mediante agentes de IA, realizar procesos en la computadora como crear y borrar archivos o buscar y explicar líneas de código mediante voz y manipulación de VSCode o de la terminal del computador (con medidas de seguridad como confirmaciones). Los desarrolladores y profesionales de TI usan Vibe para optimizar radicalmente su flujo de trabajo, automatizando tareas repetitivas, controlando su IDE y orquestando scripts complejos mediante la voz, lo que reduce la carga cognitiva y acelera el ciclo de desarrollo.

- **Personas con Discapacidades Motoras:** Vibe representa una herramienta de accesibilidad fundamental y revolucionaria, ofreciéndoles un control total sobre su entorno digital a través de la voz. Supera las barreras de los periféricos tradicionales y fomenta una mayor independencia. Ahora cualquier persona puede programar o emprender a pesar de dificultades físicas, teniendo también como apoyo una inteligencia artificial avanzada (LLM) para asistirte con tus dudas o instruirte para realizar una infinidad de tareas que el mundo actual de la IA, aunque fragmentado y no tan reconocido por el público general, hace posibles. La idea es que con Vibe como tu asistente, no sientas límites sobre las cosas que te gustaría hacer, animando a las personas a cumplir sus metas y rompiendo las barreras del autosabotaje.
- **Emprendedores y Creativos No Técnicos:** Les otorga el poder de crear agentes para automatizar tareas de negocio o creativas sin saber programar. Vibe se convierte en su "socio técnico" personal para dar vida a sus ideas, y, en una visión futura, asistir en la mayoría de procesos empresariales, incluyendo datos, insumos, reclutamiento, logística, facturación; todo orquestado y monitoreado en un solo lugar, y en un elemento vestible y portable.
- **Adultos Mayores y Usuarios Casuales:** Maximiza la simplificación de la interacción con la tecnología. Tareas como gestionar su calendario, controlar dispositivos domésticos inteligentes o simplemente usar la computadora se vuelven accesibles a través de una simple conversación.
- **Educadores y Estudiantes:** Funciona como una plataforma de aprendizaje interactiva, permitiendo explorar conceptos de IA y programación de una manera tangible y atractiva, fomentando la experimentación y la curiosidad.

Objetivos del Proyecto

1. **Objetivo General:** Desarrollar una plataforma ciberfísica experimental y con elementos reprogramables, controlada por voz, que demuestre la viabilidad de orquestar agentes de inteligencia artificial para simplificar y automatizar tareas digitales complejas.
2. **Objetivos Específicos:**

- a. **Diseñar y ensamblar un nodo de hardware (Vibe Node)** capaz de capturar audio de manera confiable, procesar una palabra de activación de forma local y proporcionar retroalimentación al usuario a través de salidas audiovisuales.
- b. **Implementar un servidor backend** que utilice un grafo de agentes (LangGraph) para interpretar la intención del usuario y delegar las tareas a los módulos de software especializados correspondientes.
- c. **Desarrollar una interfaz conversacional fluida** que permita al usuario invocar agentes mediante comandos de voz y recibir respuestas coherentes en formato de audio, validando el ciclo completo de interacción.
- d. **Habilitar una base para la interacción con el entorno de desarrollo**, explorando la capacidad del sistema para controlar aplicaciones externas como la terminal o el editor de código.

Requisitos Iniciales

1. Lista breve de lo que el sistema debe lograr:

Requisito 1 El sistema debe ser capaz de activarse mediante una palabra clave (*wake word*) detectada localmente en el Vibe Node y proveer retroalimentación sobre su estado (ej. "escuchando", "procesando") a través de señales audiovisuales.

Requisito 2 El sistema deberá procesar los comandos de voz del usuario para identificar su intención y enrutar la solicitud a un agente de software especializado para su ejecución.

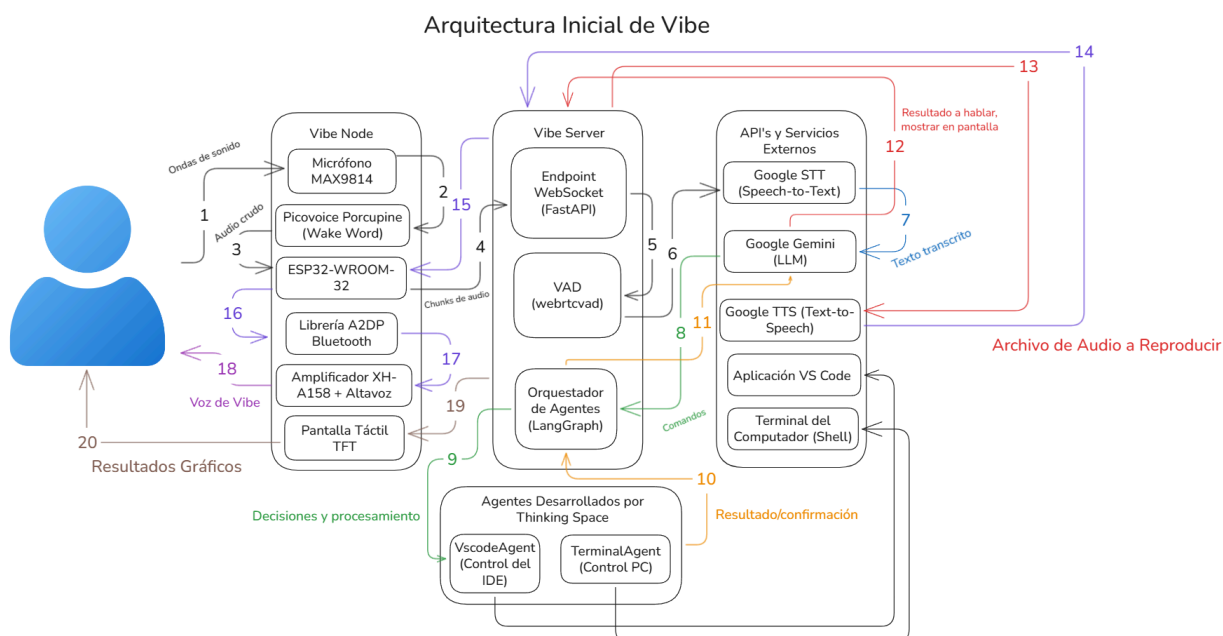
Requisito 3 El sistema debe poder completar un ciclo conversacional completo: ejecutando una acción delegada por un agente (como una consulta de información o un comando simple) y devolviendo una respuesta audible al usuario a través del altavoz del Vibe Node.

Requisito 4 El sistema debe demostrar la capacidad de interactuar con el entorno de desarrollo local, ejecutando al menos una acción tangible a través de la terminal del sistema o del editor de código (VS Code).

Diseño Preliminar del Sistema

4. Arquitectura inicial (diagrama):

A continuación se presenta un diagrama que ilustra la arquitectura de alto nivel del sistema Vibe. Este esquema visual muestra los componentes principales, desde el Vibe Node físico hasta los servicios en la nube, y la manera en que se interconectan para procesar un comando de voz de principio a fin.



Como se observa en el flujo, la arquitectura está diseñada como un ciclo completo, donde la voz del usuario inicia una cadena de eventos que involucra hardware local, un backend inteligente y APIs externas, para finalmente devolver una respuesta física y audible. Este diseño demuestra la naturaleza integrada y ciberfísica del proyecto.

5. Componentes previstos:

- a. **Microcontrolador:** El componente central del Vibe Node es la placa de desarrollo IDEABoard (basada en ESP32-WROOM-32) de CRCibernética, proporcionada por la Universidad Cenfotec. Elegimos este controlador por varias razones clave que lo hacen perfecto para este proyecto. Su procesador de doble núcleo le permite hacer dos cosas a la vez: un núcleo se dedica a escuchar la palabra de

activación mientras el otro maneja el audio y la conexión a internet, haciendo que el sistema sea rápido y no se trabe. Además, ya incluye Wi-Fi y Bluetooth de fábrica, lo que simplifica mucho el diseño y permite que se conecte a otros dispositivos sin componentes extra. Finalmente, al ser tan popular, tiene un gran ecosistema de librerías y soporte, lo que nos ayuda a integrar funciones avanzadas de forma más rápida y confiable.

b. **Sensores, actuadores y componentes de soporte:**

- i. **Módulo analógico con amplificador MAX9814:** Es el sensor de entrada principal, elegido por su claridad para capturar los comandos de voz del usuario y poca sensibilidad al ruido del circuito.
- ii. **Amplificador Bluetooth XH-A158 y Altavoz de 1W:** Este conjunto forma la "voz" de Vibe. El amplificador recibe la señal de audio y la potencia para ser reproducida de forma clara en el altavoz.
- iii. **TFT de 2.4" con controlador táctil ILI9341 + XPT2046:** Funciona como un componente dual. Muestra información visual y el estado del sistema (actuador) y a la vez permite una interacción directa por parte del usuario (sensor), aumentando la ciberfisidad del sistema.
- iv. **Convertidor de Voltaje XL6009:** Es una pieza de soporte clave. Aumenta el voltaje del sistema a 5V para alimentar correctamente el amplificador de audio, garantizando su funcionamiento óptimo.
- v. **Batería LiPo y Módulo de Carga TP4056:** Proporcionan la energía necesaria para que el Vibe Node sea completamente portátil y seguro, gestionando la carga y protegiendo la batería.

c. **LLM/API:**

- i. **Google Gemini 2.5 Flash:** Utilizado como el cerebro central para la comprensión del lenguaje natural (NLU) y la clasificación de intenciones dentro del orquestador de agentes.

- ii. **Google Cloud Speech-to-Text:** Para la transcripción precisa de los comandos de voz del usuario a texto.
- iii. **Google Cloud Text-to-Speech:** Para convertir las respuestas generadas por el sistema en una voz natural y audible.
- iv. **VS Code API:** Una interfaz de programación de la extensión de VS Code, ambos hechos por nosotros, servirá como agente de programación “VscodAgent”. Se interactúa con ella a través de nuestro WebSocket para que el “VscodAgent” pueda manipular archivos, escribir código y controlar el editor.
- v. **Interfaz de Terminal (Shell):** El “TerminalAgent” interactúa directamente con la shell del sistema operativo (Bash, PowerShell, etc.) para ejecutar comandos locales como git, npm o cualquier script.
- vi. **Google Workspace APIs:** Para la expansión de agentes se contempla el uso de APIs como las de **Google Calendar** y **Gmail**, permitiendo a Vibe gestionar eventos y correos electrónicos.

d. **Librerías y herramientas:**

- i. **FastAPI (Python):** El framework para construir el servidor backend por su alto rendimiento y moderna sintaxis asíncrona.
- ii. **Uvicorn (Python):** Servidor ASGI (Asynchronous Server Gateway Interface) de alto rendimiento para ejecutar FastAPI.
- iii. **webrtcvad-wheels (Python):** Para la Detección de Actividad de Voz (VAD) en el servidor, determinando cuándo el usuario ha terminado de hablar..
- iv. **LangGraph (Python):** La librería clave para modelar y ejecutar el grafo de agentes, permitiendo una lógica de control compleja y extensible. Utilizado en el servidor.
- v. **Picovoice Porcupine:** El motor para la detección de la palabra de activación (*wake word*) haciendo uso de inteligencia artificial en el borde

del sistema ciberfísico, seleccionado por su alta precisión y eficiencia para ejecutarse directamente en el ESP32.

- vi. **PlatformIO (C++)**: El entorno de desarrollo para programar el firmware del ESP32, gestionando las librerías de hardware de forma profesional.
- vii. **pschatzmann A2DP Library (C++)**: Librería para el ESP32 que habilita la transmisión de audio de alta calidad a través de Bluetooth (A2DP). Utilizado para comunicar el audio que devuelve el servidor del ESP32 hacia el amplificador bluetooth, para así reproducir el audio que devuelve el servidor en el Vibe Node.
- viii. **WebSockets**: Protocolo de comunicación entre el Vibe Node y servidor principal, provee una conexión bidireccional que se mantiene abierta hasta que uno de los dos dispositivos abandona la conexión, esto provee mejor eficiencia que utilizar otros protocolos como HTTP, cuya conexión no es persistente y es unidireccional, aumentando la complejidad e ineficiencia.

6. Bocetos o esquemas:

La inspiración estética y conceptual para el brazalete Vibe nace de la tecnología vestible icónica en la cultura popular. A continuación se presenta una imagen del 'Pip-Boy', el asistente personal de muñeca del videojuego Fallout, que sirve como nuestra principal referencia creativa para el diseño físico del Vibe Node.



El objetivo no es replicar el diseño, sino capturar su espíritu: un dispositivo funcional que se sienta como una herramienta potente y personal en la muñeca. Vibe busca modernizar este concepto, cambiando los botones y menús rústicos por una interacción fluida basada en la voz y una pantalla táctil.

Nota: El "Pip-Boy" y la saga "Fallout" son marcas registradas de Bethesda Softworks. Su imagen se utiliza en este documento únicamente como referencia conceptual e inspiración de diseño, reconociendo en todo momento a sus respectivos dueños.

Plan de Trabajo

- **Cronograma preliminar:**

A continuación se presenta el plan de trabajo para desarrollar el prototipo de Vibe. El cronograma se divide en ocho hitos clave, concentrados en un sprint de desarrollo intensivo desde el 3 hasta el 19 de agosto, para guiar la construcción del proyecto de manera organizada y eficiente.

Hito	Objetivos Clave	Fechas Estimadas

1	Server Uvicorn/FastAPI y versión preliminar de VscodAgent.	23 Ago - 26 Ago
2	Montaje físico de micrófono y ESP32. Lograr flujo completo de entrada de voz y respuesta de IA impresa en consola.	26 Ago - 30 Ago
3	Implementar Wake Word y altavoz. Lograr flujo completo con activación por voz y respuesta hablada.	1 Ago - 5 Ago
4	Implementar LangGraph y el TerminalAgent. Probar la integración con un VscodAgent más avanzado.	5 Ago - 8 Ago
5	Integrar la pantalla táctil y la batería. Diseñar la carcasa del brazalete en software CAD.	8 Ago - 11 Ago
6	Implementar APIs de Google (reloj/calendario) y explorar los comandos reprogramables (stretch goal).	11 Ago - 13 Ago
7	Con la carcasa impresa, realizar el montaje y ensamblaje final de todos los componentes del brazalete.	13 Ago -14 Ago
8	Documentación final, pruebas exhaustivas y desarrollo de nuevas funciones si el tiempo lo permite.	14 Ago - 19 Ago

Este plan de trabajo está diseñado para enfocar nuestros esfuerzos en los objetivos más importantes de cada fase. Siguiendo estos hitos, aseguraremos la entrega de un prototipo funcional, integrado y listo para su demostración en la fecha límite establecida.

- **Riesgos identificados y mitigaciones:**

A continuación se presentan los riesgos identificados del proyecto con su respectiva descripción y mitigación, están divididos por categorías que cubren los diversos niveles del proyecto.

Riesgos de Hardware y Ensamblaje

- **Riesgo: Insuficiencia de Cómputo del ESP32-WROOM-32.**
 - **Descripción:** El microcontrolador debe gestionar simultáneamente el motor de *wake word*, la captura y codificación de audio crudo en formato específico, la conexión Wi-Fi y la lógica de la pantalla. Existe el riesgo de que la carga de trabajo sature la CPU, provocando tartamudeo en el audio, pérdida de paquetes o fallos en la detección de la palabra clave.
 - **Mitigación:** Utilizar librerías ligeras e implementar código eficiente, hacer streaming en medida de lo posible, usar funciones asíncronas del lado del servidor, y dejar las cargas de procesamiento más pesadas al servidor.
- **Riesgo: Ruido Eléctrico en la Cadena de Audio.**
 - **Descripción:** La actividad del Wi-Fi y de los controladores de la pantalla puede introducir interferencia en la señal del micrófono o en la salida del amplificador, degradando la calidad del audio para el STT y la respuesta audible.
 - **Mitigación:** Probar constantemente la calidad del micrófono y en diversos ambientes (en multitud, con ventiladores encendidos, etc.), asegurar un mecanismo de VAD (Voice Activity Detection) que no se base en detectar silencios absolutos para enviar un chunk de audio, sino que detecte el silencio distintivo entre una voz clara hablando y el ruido ...
- **Riesgo: Complejidad del Montaje Físico.**

- **Descripción:** Integrar la placa, batería, pantalla, altavoz, amplificador y micrófono en un chasis compacto impreso en 3D presenta desafíos de espacio, gestión de cables y disipación de calor. Un montaje deficiente puede llevar a cortocircuitos o sobrecalentamiento.
- **Mitigación:** Diseñar el case de forma modular en software CAD, permitiendo iteraciones rápidas. Asegurar ventilación adecuada y utilizar separadores y montajes específicos para cada componente para garantizar un ensamblaje limpio, eficiente y seguro.

Riesgos de Software e Inteligencia Artificial

- **Riesgo: Latencia Acumulada en el Ciclo Conversacional.**
 - **Descripción:** El tiempo de respuesta total es la suma de múltiples latencias: detección local, streaming de audio, STT, inferencia del LLM en LangGraph, TTS y el streaming de audio de vuelta. Una suma elevada (>3-4 segundos) romperá la sensación de una conversación fluida.
 - **Mitigación:** Utilizar streaming en cada etapa posible. Optimizar los prompts para que Gemini genere respuestas rápidas. Usar modelos de IA más veloces si es necesario (ej. Gemini 2.5 Flash). El Vibe Node debe dar feedback visual *instantáneo* en cada paso para gestionar la latencia percibida por el usuario.
- **Riesgo: Precisión y Falsos Positivos de la Wake Word.**
 - **Descripción:** A pesar de su calidad, el motor de *wake word* puede activarse con palabras o ruidos similares (falso positivo) o no activarse cuando se le llama en un entorno ruidoso (falso negativo).
 - **Mitigación:** Ajustar el umbral de sensibilidad del motor de detección en el código del ESP32 hasta el deseado.
- **Riesgo: Complejidad Técnica del Grafo de Agentes.**
 - **Descripción:** La arquitectura de LangGraph, aunque potente, es inherentemente compleja. Existe el riesgo de que la gestión del estado, las transiciones

condicionales y los ciclos entre agentes introduzcan bugs lógicos difíciles de depurar.

- **Mitigación:** Empezar con un grafo lineal y simple. Utilizar herramientas como LangSmith para trazar y visualizar la ejecución de los agentes. Añadir nuevos agentes y transiciones de forma incremental, con pruebas unitarias para cada uno.

Riesgos de Integración y Proyecto

- **Riesgo: Fragilidad por Múltiples Dependencias Tecnológicas.**

- **Descripción:** El proyecto depende de C++ (PlatformIO), Python (FastAPI), múltiples APIs de Google y potencialmente JavaScript (extensión VS Code). Una actualización o cambio en la API de cualquiera de estos componentes puede romper la integración de todo el sistema.
- **Mitigación:** Fijar las versiones de todas las dependencias en los archivos de configuración (`requirements.txt`, `platformio.ini`). Definir contratos de API claros y versionados entre el servidor y los clientes (Vibe Node, extensión).

- **Riesgo: Sobrecarga de Complejidad del Proyecto.**

- **Descripción:** Como equipo, estamos utilizando este proyecto como una oportunidad para experimentar con tecnologías y conceptos que son nuevos o poco explorados por nosotros. El desconocimiento natural de herramientas como LangGraph, la falta de conocimientos de electrónica, la programación de sistemas ciberfísicos y librerías específicas de ESP32 puede generar atrasos, incompatibilidades o cambios al plan original, poniendo en riesgo la calidad y el alcance de la entrega final.
- **Mitigación:** Se implementará un flujo de trabajo ágil y enfocado en la eficiencia para reducir la incertidumbre rápidamente:
 - **Optimización del Tiempo:** Se trabajará en paralelo. Mientras un miembro del equipo desarrolla un componente cuya tecnología ya se domina, otro

puede investigar y realizar pruebas de concepto simples para la siguiente tecnología desconocida.

- **Partir de Ejemplos Funcionales:** Antes de escribir código propio para una nueva función, se buscará y replicará un ejemplo oficial o de la comunidad que ya funcione. Esto nos dará una base sólida y probada para construir encima.
- **Apalancamiento en IA:** Se usarán intensivamente herramientas de IA para acelerar la investigación, generar código de base y depurar problemas, actuando como un multiplicador de nuestra capacidad de desarrollo.

Más que un prototipo, Vibe es la primera exploración de un futuro donde la interacción con la inteligencia artificial es tangible, intuitiva y conversacional. Nuestro objetivo es romper las barreras técnicas, transformando la compleja orquestación de agentes de software en una herramienta creativa al alcance de todos. Este proyecto es nuestro primer paso para convertir esa visión en una realidad funcional.